LOVELY PROFESSIONAL UNIVERSITY

School of Computer Science & Engineering

PROJECT REPORT

CivicFix

Crowdsourced Civic Issue Reporting and Resolution System

Student Name	Prashant Baral
Registration Number	12327401
Roll Number	65
Section	K3O2307
Term	26271
Course Code	K4E2338
Faculty	Dr. Manpreet Singh Sehgal
Submission Date	29/10/26

Student Declaration

I declare that this project report is prepared for academic submission and that sources used in the report are acknowledged. The descriptions of implementation, dataset provenance, and test results should be read with the stated limitations. Any placeholders must be completed and the final report reviewed for accuracy before submission.

Student signature: Prashant baral

Abstract

CivicFix is a civic issue reporting and tracking system. Citizens report problems such as potholes, garbage, broken streetlights, water-supply faults and blocked drains, and officials move each report through three stages: `submitted`, `in_progress` and `resolved`. The application uses Python and FastAPI for the web interface, PyMongo to talk to MongoDB directly, and Swagger UI for testing in the browser.

The database is the main part of the project. All reports are stored in one MongoDB collection called `issues`. Each report is a single document that contains its own location and its complete status history. The collection is protected by a server-side validation rule and six indexes, and status changes are made with one atomic update so that a report can only move forward. Aggregation pipelines produce summaries by category, status, priority and city, and `explain()` is used to show how indexes reduce the work MongoDB has to do.

The sample file holds 58 fictional reports from seven cities in Punjab. The project is a local academic prototype. User login, notifications are not implemented and are listed under future enhancements.

1. Introduction

1.1 Background

People meet civic problems every day: potholes on the road, garbage bins that are not emptied, streetlights that do not work, and water supply that stops without notice. These complaints are usually made by phone, in writing or on messaging apps, so each one is recorded in a different place. It becomes hard to know how many complaints are still pending, which problems are most urgent, and how long a fix takes.

CivicFix keeps every complaint in one database with a clear status. This supports **Sustainable Development Goal 11, Sustainable Cities and Communities**, by making the handling of local problems more organised and transparent.

1.2 Aim

To build a working civic complaint tracker that uses MongoDB as its only data store and demonstrates document modelling, validation, indexing, CRUD operations, aggregation and query optimisation.

1.3 Objectives

- Store each civic report as one MongoDB document in a collection named `issues`, with the location and status history embedded inside it.
- Provide a REST interface to create, view, update and delete reports, with filters, text search and pagination.
- Allow only forward status changes (`submitted` → `in_progress` → `resolved`) and keep a dated history of every change.
- Check data in two places: in the API (Pydantic) and inside MongoDB (`$jsonSchema`).
- Show summaries such as reports by category, status, priority and city, and the average time taken to resolve a report.
- Create indexes that match the queries used, and confirm their benefit with `explain()`.

1.4 Scope and boundaries

This is a local academic prototype. The 58 reports in `data.csv` are fictional. User login, notifications, photo upload, map views and public hosting are not implemented.

2. Problem Statement and Proposed Solution

2.1 Problem statement

Without a central record, a municipal office cannot easily answer simple questions: how many reports are still open, which type of problem is reported most, which city has the most complaints, and how long a pothole takes to fix. A database-backed tracker that records each report along with its progress can answer these questions directly.

2.2 Proposed solution

CivicFix provides a REST API built with FastAPI. A request is checked by Pydantic, converted into a PyMongo operation and stored in the `issues` collection of the `civicfix` database. A second group of endpoints runs aggregation pipelines for summaries, and one endpoint returns `explain()` results so that the effect of indexes can be shown.

2.3 Benefits and limitations

- **Benefits:** one place for all reports; filterable and searchable lists; a dated history for every report; ready-made summaries; and a practical demonstration of MongoDB features.
- **Limitations:** the data is fictional; there are no user accounts, so anyone can call any endpoint; and the application does not yet send any notifications.

3. Technology Stack

Technology	Role in CivicFix
MongoDB	Document database. Database `civicfix`, collection `issues`.
PyMongo	Python driver used directly, with no ORM or ODM.
Python 3	Application logic and query building.
FastAPI	Web routes, automatic checking of input and the Swagger UI page at `/docs`.
Uvicorn	Server that runs the FastAPI application.
Pydantic	Defines the shape of requests and responses and the allowed values.
pydantic-settings	Reads the database address from a local `.env` file.
mongosh	MongoDB shell used to check the data and produce the terminal outputs in this report.
curl	Command-line tool used to try the API endpoints.

3.1 How the components work together

The client (Swagger UI or `curl`) sends a request to FastAPI. Pydantic checks the request. A service function builds the MongoDB query and sends it through PyMongo. MongoDB runs it, checks any new data against its validation rule, and returns the result as JSON.

Client (Swagger UI / curl) → FastAPI routes → Service layer + PyMongo → MongoDB (`civicfix.issues`) → JSON response

4. System Architecture and Workflow

4.1 Main components

File	What it does
`app/main.py`	Starts the FastAPI app. On startup it creates the collection and indexes automatically.
`app/config.py`	Reads the connection settings from the environment.
`app/database.py`	Holds the database connection, the validation rule and the index definitions.
`app/models.py`	Pydantic models and the allowed values for category, priority and status.
`app/routes/issues.py`	Endpoints under `/issues`: create, read, update, delete and status change.
`app/routes/analytics.py`	Endpoints under `/analytics`: summaries and `explain()`.
`app/services/crud.py`	MongoDB queries for create, read, update and delete, and the atomic status change.
app/services/analytics.py	The aggregation pipelines and the explain() comparison.

4.2 Reporting workflow

1. A citizen sends `POST /issues` with the title, category, description, priority and location.
2. Pydantic checks the data. The service adds the status `submitted`, the first history entry and the time stamps.
3. MongoDB checks the document against its validation rule and stores it.
4. An official sends `PATCH /issues/{id}/status`. One atomic update checks the current status, changes it and adds a history entry.
5. If the change is not allowed, nothing is updated and the API returns HTTP 409.

4.3 Analytics workflow

1. The client calls an `/analytics` endpoint.
2. The service runs an aggregation pipeline on the `issues` collection.
3. MongoDB groups, counts and averages the reports and returns only the summary.
4. The API sends the summary back as JSON.

5. MongoDB Database Design

5.1 Database and collection

The database is named `civicfix` and contains one collection, `issues`. Every query in the project is about reports, so a second collection would only add extra lookups.

Item	Value
Database	`civicfix`
Collection	`issues`
Documents loaded	58 (from `data.csv`)
Indexes	7 (the default `_id` index plus 6 created by the application)
Validation	`$jsonSchema`, level strict, action error

5.2 Example report document

This is one real document from the sample data. Notice that `location` is a small object and `status_history` is a list inside the same document.

```
{
  title: "Tree fallen on road near Chaura Bazar",
  category: "other",
  description: "A large tree has fallen on road near Chaura Bazar blocking road”
  priority: "urgent",
  status: "in_progress",
  location: { area: "Chaura Bazar", city: "Ludhiana", latitude: 30.905, longitude: 75.8545 },
  status_history: [
    { status: "submitted",   changed_at: ISODate("2026-10-29T13:51:43Z"), note: "Report
created" },
    { status: "in_progress", changed_at: ISODate("2026-10-29T15:51:43Z"), note: "Tree removal
team dispatched" }
  ],
  created_at: ISODate("2026-10-29T13:51:43Z"),
  updated_at: ISODate("2026-10-29T15:51:43Z")
}
```

5.3 Design decisions

Decision	Reason
Embed `location`	It belongs to one report and is always read with it, so no join is needed.
Embed `status_history` as a list	A report has at most three entries. One update can change the status and add the history entry together.
One collection	All queries are about reports.
Fixed lists of values for category, priority and status	Keeps the data clean and makes counting reliable.

Decision	Reason
Latitude and longitude may be empty	Seven sample reports have no coordinates.

5.4 Data types used

Field	BSON type
`_id`	ObjectId
`title`, `description`, `category`, `priority`, `status`	String
`location`	Embedded object
`latitude`, `longitude`	Double (or null)
`status_history`	Array of embedded objects
`created_at`, `updated_at`, `changed_at`	Date

5.5 Modelling trade-off

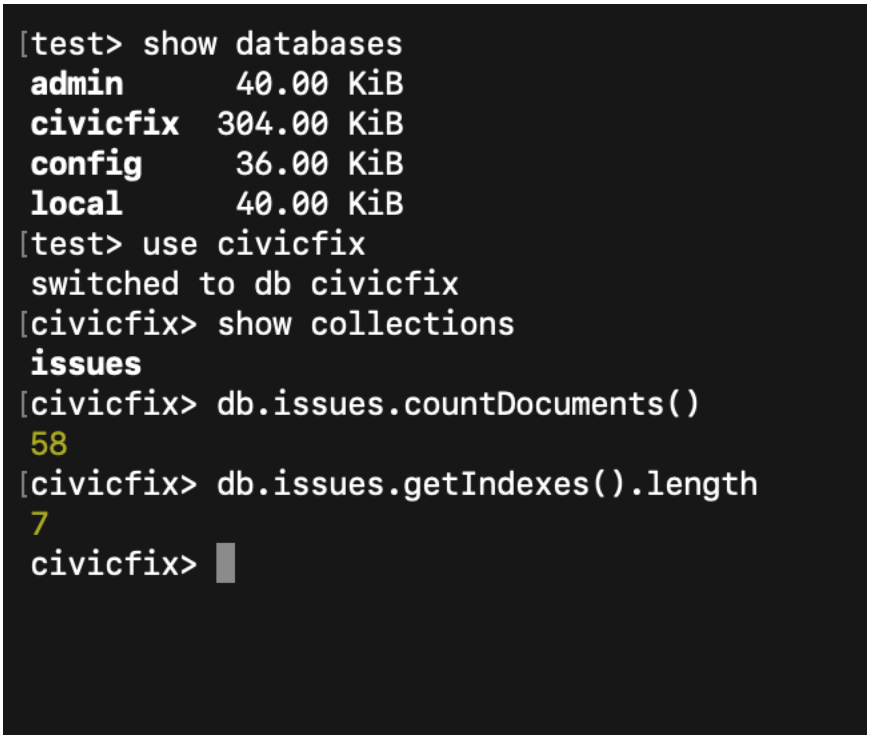
Embedding keeps a report and its history together, which makes reading and updating simple and fast. The trade-off is that a document grows as data is added. This is safe here because a report can change status only twice after it is created. If reports could collect unlimited comments, a separate collection would be a better choice.

6. Sample Dataset

The file `data.csv` contains 58 fictional civic reports from seven cities in Punjab. Each row has 13 columns: id, title, category, priority, status, city, area, latitude, longitude, created_at, updated_at, latest_note and description. The file is an export of the sample reports from the database. Each row maps to the document shown in section 5.2: the city, area, latitude and longitude form the location object, and the status, the two dates and the latest note form the status history.

Category	Reports	Status	Reports	City	Reports
pothole	14	in_progress	25	Phagwara	14
garbage	10	resolved	23	Ludhiana	12
drainage	9	submitted	10	Jalandhar	12
water_supply	8			Amritsar	6
streetlight	7			Patiala	5
public_facility	6			Hoshiarpur	5
other	4			Kapurthala	4

The terminal check below shows the database after the 58 reports were loaded. It confirms that the database `civicfix` has one collection, 58 documents and 7 indexes.



7. Data Validation

Validation means MongoDB checks every report before saving it and refuses any report that breaks the rules. This works even when someone bypasses the application and types directly in `mongosh`.

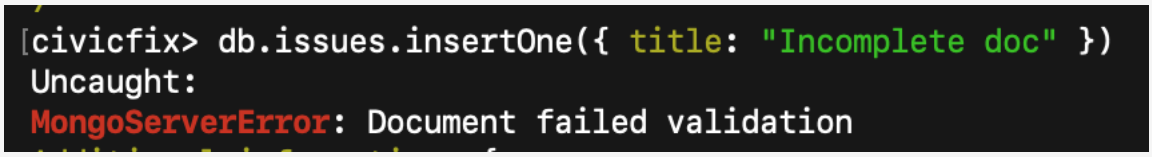
Field	What is allowed
Required fields	title, category, description, priority, status, location, status_history, created_at, updated_at
`title`	Text of 3 to 120 characters
`description`	Text of 10 to 2000 characters
`category`	pothole, garbage, streetlight, water_supply, drainage, public_facility or other
`priority`	low, medium, high or urgent
`status`	submitted, in_progress or resolved
`location`	Object with area and city (at least 2 characters); latitude between -90 and 90; longitude between -180 and 180
`status_history`	List with at least one entry

The rules are written once, when the collection is created. The application does this automatically at startup; the same command can be typed in `mongosh`.

7.1 Bad reports are refused

Two bad reports were tried directly in `mongosh`. The first has only a title. The second is complete except that `alien_invasion` is not an allowed category. Both were refused, and the count stayed at 58.

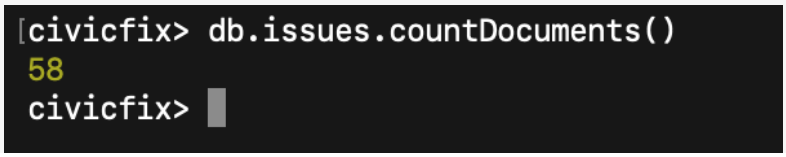
```
1. civicfix> db.issues.insertOne({ title: "Incomplete doc" })
   MongoServerError: Document failed validation
```




```
2. civicfix> db.issues.insertOne({ ..., category: "alien_invasion", ... })
   MongoServerError: Document failed validation
```




```
3. civicfix> db.issues.countDocuments()
58
```



7.2 Two layers of checking

- **Layer 1, the API:** Pydantic checks each request and returns a clear HTTP 422 message to the user.
- **Layer 2, the database:** the `$jsonSchema` rules protect the data from any other source.

8. Indexing and Query Optimisation

An index works like the index of a book. Without one, MongoDB reads every report to find the matches. With one, it jumps straight to them.

8.1 Indexes created

Index	Built on	Makes this fast
`status_created_at`	status, then newest first	Reports with a given status
`category_created_at`	category, then newest first	Reports of a given category
`priority_created_at`	priority, then newest first	Reports of a given priority

Index	Built on	Makes this fast
`city_created_at`	city, then newest first	Reports from a given city
`created_at_desc`	newest first	The plain newest-first list
`text_search`	title and description	Searching for words; a match in the title counts three times as much

The six indexes are created with one command each:

```
db.issues.createIndex({ status: 1, created_at: -1 },          { name: "status_created_at" })
db.issues.createIndex({ category: 1, created_at: -1 },        { name: "category_created_at" })
db.issues.createIndex({ priority: 1, created_at: -1 },        { name: "priority_created_at" })
db.issues.createIndex({ "location.city": 1, created_at: -1 }, { name: "city_created_at" })
db.issues.createIndex({ created_at: -1 },                      { name: "created_at_desc" })
db.issues.createIndex({ title: "text", description: "text" },
                      { name: "text_search", weights: { title: 3, description: 1 } })
```

In each line, `1` means ascending order and `-1` means newest first.

8.2 Why each index also includes `created_at`

The list always shows the newest reports first. An index on the city alone finds the right reports but leaves MongoDB to sort them afterwards, and the query planner may ignore the index. An index on city and `created_at` already stores the reports in that order, so no extra sorting is needed.

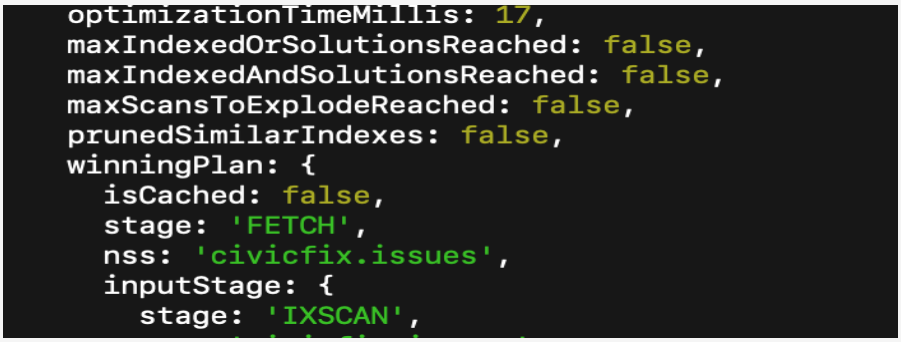
8.3 Proving it with explain()

The same query (all reports from Phagwara, newest first) was run twice: once normally, and once forced to read every document.

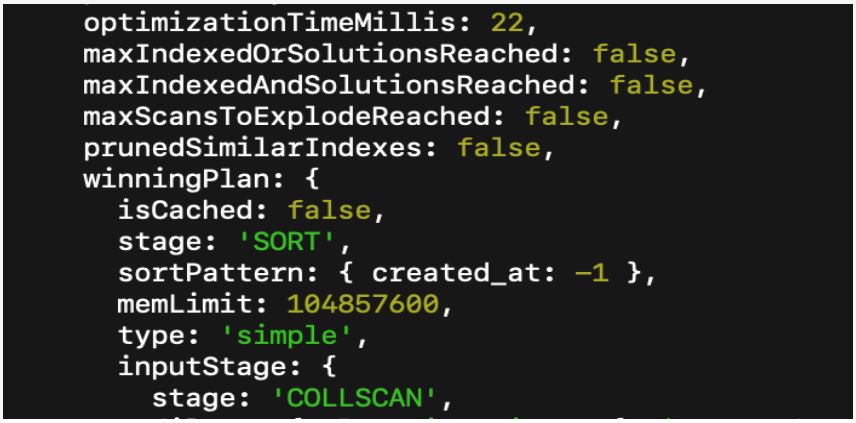
Run	Method	Reports examined	Reports returned
Normal	Index scan on `city_created_at`	14	14
Forced	Read everything, then sort	58	14

The two runs use the same query. The second one adds `hint({ $natural: 1 })`, which tells MongoDB to ignore the indexes and read every document.

```
// Normal run: MongoDB uses the index
db.issues.find({ "location.city": "Phagwara" })
         .sort({ created_at: -1 })
         .explain("executionStats")
```



```
// Forced run: MongoDB reads the whole collection
db.issues.find({ "location.city": "Phagwara" })
         .sort({ created_at: -1 })
         .hint({ $natural: 1 })
         .explain("executionStats")
```



With only 58 reports the difference in time is too small to measure, so the number of reports examined shows the saving. The saving grows as the collection grows. The same comparison is available in the API at `GET /analytics/explain`.

9. Searching, Filtering and Pagination

The list endpoint `GET /issues` supports these options. Filters combine with AND.

Option	Purpose
`category`, `status`, `priority`	Show only reports with that value
`city`	Show only reports from that city
`q`	Search words in the title and description, best match first
`sort`	Newest first (default) or oldest first
`page`, `page_size`	Choose the page and how many reports to show per page

For example, potholes in Phagwara, newest first, return three reports: "Road cave-in near Hargobind Nagar" (urgent, in progress), "Large pothole near main road" (high, submitted) and "Pothole near Bus Stand Phagwara" (urgent, resolved). A search for "streetlight" returns only the reports that contain that word, newest first in the API. In mongosh, sorting by text score puts the best matches first, because a match in the title counts three times as much as one in the description.

The same operations in `mongosh`:

```
// CREATE: save a new report
db.issues.insertOne({
  title: "Broken streetlight outside Model Town gate",
  category: "streetlight",
  description: "The streetlight outside the gate has stopped working.",
  priority: "medium",
  status: "submitted",
  location: { area: "Model Town", city: "Phagwara" },
  status_history: [ { status: "submitted", changed_at: new Date(), note: "Report
```

```
created" } ],
  created_at: new Date(),
  updated_at: new Date()
})

// READ: potholes in Phagwara, newest first, showing only three fields
db.issues.find(
  { category: "pothole", "location.city": "Phagwara" },
  { title: 1, priority: 1, status: 1, _id: 0 }
).sort({ created_at: -1 })

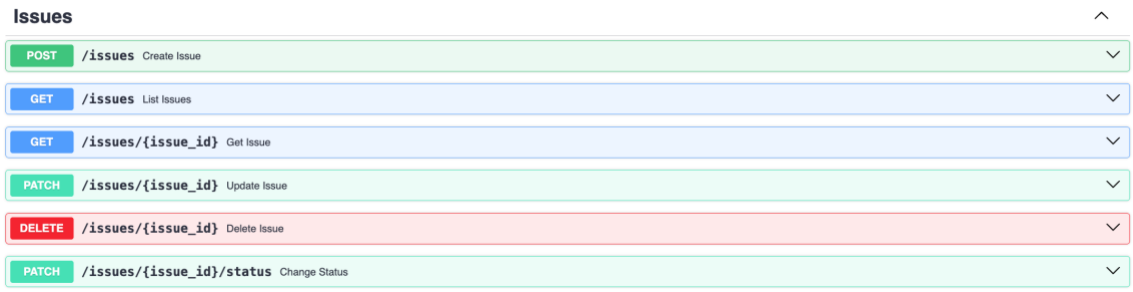
// SEARCH: reports that mention a word
db.issues.find({ $text: { $search: "streetlight" } })

// PAGES: page 3 with 5 reports per page (skip 10, show 5)
db.issues.find().sort({ created_at: -1 }).skip(10).limit(5)

// UPDATE: change only the priority
db.issues.updateOne(
  { title: "Broken streetlight outside Model Town gate" },
  { $set: { priority: "high", updated_at: new Date() } }
)

// DELETE: remove the report
db.issues.deleteOne({ title: "Broken streetlight outside Model Town gate" })
```

Basic create, read, update and delete operations were also tested in `mongosh` with one test report. After the report was created, changed and deleted, the count returned to 58.



10. Status Workflow

A report can only move forward, one step at a time. It cannot skip a step or go back.

Current status	Allowed next status
`submitted`	`in_progress`
`in_progress`	`resolved`
`resolved`	none (final)

10.1 How it works

The status change is done with one MongoDB command, `findOneAndUpdate`. The command first looks for the report **only if** its current status allows the move. If it finds the report, it changes the status and adds a line to `status_history` in the same step. If it finds nothing, nothing changes and the API returns HTTP 409. Because the check and the change happen together, two people acting at the same moment cannot break the order.

```
db.issues.findOneAndUpdate(
  { _id: reportId, status: "submitted" },          // only if still submitted
  {
    $set:  { status: "in_progress" },
    $push: { status_history: { status: "in_progress", changed_at: new Date(), note:
"Crew assigned" } }
  }
)
```

10.2 Result

A test report was moved from `submitted` to `in_progress` and then to `resolved`. Trying the first move a second time returned `null` because the report was no longer `submitted`. After the last step, the report held three dated history entries, which form its audit trail and are also used to calculate resolution time.

11. Aggregation and Analytics

An aggregation pipeline is like an assembly line. The reports pass through stages one after another, and each stage filters, groups or reshapes them until the answer comes out.

Stage	What it does
`$match`	Keeps only the reports that fit a condition
`$group`	Groups reports and counts or averages each group
`$sort`	Orders the results
`$project`	Chooses and calculates the fields to show
`$facet`	Runs several summaries side by side in one pass

For example, this pipeline counts reports per category:

```
db.issues.aggregate([
  { $group: { _id: "$category", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])
```

```
civicfix> db.issues.aggregate([
|   { $group: { _id: "$category", count: { $sum: 1 } } },
|   { $sort: { count: -1 } }
| ])
[|
[
  { _id: 'pothole', count: 14 },
  { _id: 'garbage', count: 10 },
  { _id: 'drainage', count: 9 },
  { _id: 'water_supply', count: 8 },
  { _id: 'streetlight', count: 7 },
  { _id: 'public_facility', count: 6 },
  { _id: 'other', count: 4 }
]
civicfix>
```

The `/analytics/summary` endpoint uses `$facet` to return the total and the counts by category, status, priority and city in a single pass. The `/analytics/resolution-time` endpoint keeps resolved reports, works out the hours between creation and the last history entry, and averages them per category.

The two longer pipelines look like this:

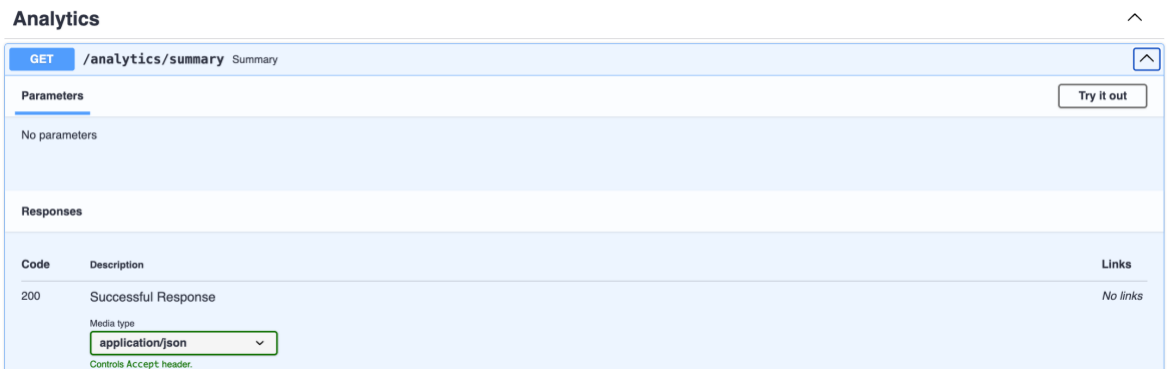
```
// Several summaries in one pass ($facet)
db.issues.aggregate([{ $facet: {
  total:       [ { $count: "n" } ],
  by_status:   [ { $group: { _id: "$status",   count: { $sum: 1 } } } ],
  by_priority: [ { $group: { _id: "$priority", count: { $sum: 1 } } } ]
} }])

// Average hours to resolve, per category
db.issues.aggregate([
  { $match: { status: "resolved" } },                       // only resolved reports
  { $project: { category: 1, hours: { $divide: [            // hours between creation
      { $subtract: [ { $last: "$status_history.changed_at" }, "$created_at" ] },
      3600000 ] } } },                                       // and the last status
change
  { $group: { _id: "$category", avg_hours: { $avg: "$hours" } } },
  { $sort: { avg_hours: -1 } }                              // slowest first
])
```

11.1 Results on the 58 sample reports

Question	Answer
Most reported category	Pothole, 14 reports (garbage 10, drainage 9, water supply 8, streetlight 7, public facility 6, other 4)
Reports by status	In progress 25, resolved 23, submitted 10
Reports by priority	Urgent 22, high 20, medium 13, low 3
City with most reports	Phagwara 14, Jalandhar 12, Ludhiana 12, Amritsar 6, Hoshiarpur 5, Patiala 5, Kapurthala 4
Average hours to resolve	Public facility 351.6, other 264, water supply 216, drainage 216, pothole 185.1, streetlight 147

Garbage has no resolved reports in the sample, so it does not appear in the resolution-time list. The resolution times depend on the dates in the sample file, so they should be treated as illustrations of the method.

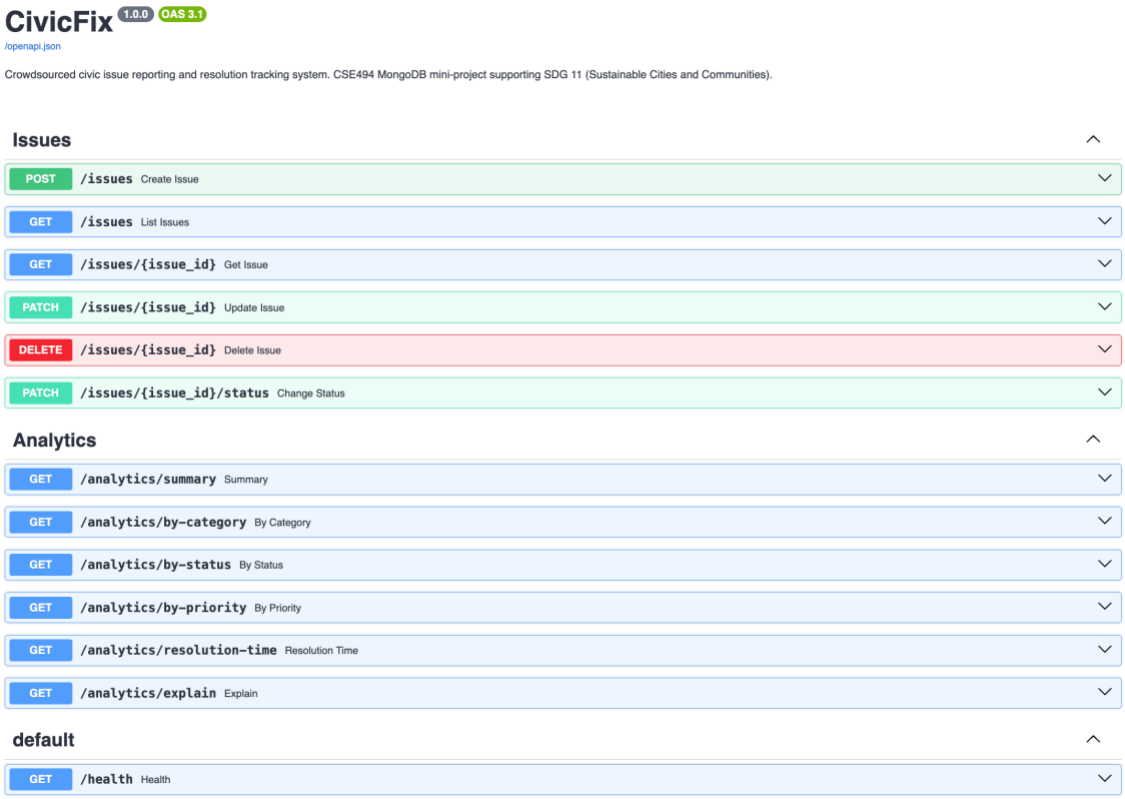


12. REST API

FastAPI exposes the database through the endpoints below. Swagger UI at `http://127.0.0.1:8000/docs` lets every endpoint be tested from the browser.

Method	Endpoint	Purpose
POST	`/issues`	Report a new civic issue
GET	`/issues`	List reports with filters, search, sorting and pages
GET	`/issues/{id}`	Get one report
PATCH	`/issues/{id}`	Change title, description, category, priority or location
PATCH	`/issues/{id}/status`	Move the status forward and add a note
DELETE	`/issues/{id}`	Delete a report
GET	`/analytics/summary`	Total plus counts by category, status, priority and city
GET	`/analytics/by-category`, `/by-status`, `/by-priority`	Counts grouped by one field
GET	`/analytics/resolution-time`	Average hours to resolve, per category
GET	`/analytics/explain`	Query plan and statistics for the index comparison
GET	`/health`	Checks that the API and MongoDB are connected

Errors are reported clearly: HTTP 400 for a badly formed id, 404 when a report is not found, 409 for a status move that is not allowed and 422 for invalid input. The database address is kept in a local `.env` file, which is excluded from source control.



13. MongoDB Concepts Demonstrated

Concept	CivicFix example	Why it matters
Document model	Each report is one BSON document	Stores a report and its history together
Embedded documents	`location` and `status_history`	No joins for related data
BSON types	ObjectId, String, Date, Double, Object, Array	The right type for each field
Schema validation	`$jsonSchema`, strict, error	The database rejects invalid data
CRUD	`insertOne`, `find`, `updateOne`, `deleteOne`	The core operations
Atomic update	`findOneAndUpdate` with `$set` and `$push`	Safe status changes
Dot notation	`location.city`	Queries a field inside an embedded object
Compound indexes	`(field, created_at)`	Filter and sort in one step
Text index	`title` and `description`, weighted	Word search over title and description
Aggregation	`$group`, `$sort`, `$facet`, `$match`,	Summaries computed inside

Concept	CivicFix example	Why it matters
	`$project`, `$avg`	MongoDB
Pagination	`sort`, `skip`, `limit`	Limits how many reports are returned at once
`explain()`	`executionStats`, `hint`	Shows what an index saves

14. Testing and Validation

The table separates what has been shown by terminal output in this report from checks that still need to be repeated on the final build.

ID	Test case	Expected result	Status
T01	Database, collection and count	`civicfix`, `issues`, 58 documents, 7 indexes	Verified in `mongosh` (section 6)
T02	Insert with missing fields	Document failed validation	Verified (section 7.1)
T03	Insert with invalid category	Document failed validation	Verified (section 7.1)
T04	Create, read, update, delete	Count returns to 58	Verified (section 9)
T05	Filter and sort	Three Phagwara potholes, newest first	Verified (section 9)
T06	Text search	Only streetlight-related reports returned	Verified (section 9)
T07	Valid status move	`submitted` to `in_progress`, history added	Verified (section 10)
T08	Repeated or skipped status move	`null` from MongoDB, HTTP 409 from the API	Verified in `mongosh`; repeat the API call on the final build
T09	Aggregation counts	Match the counts in section 6	Verified (section 11)
T10	Index versus forced scan	14 documents examined versus 58	Verified (section 8.3)
T11	Invalid id format	HTTP 400	To be run on the final build
T12	MongoDB not running	Startup or request fails with a clear error	To be run on the final build
T13	Swagger UI check of all endpoints	Every endpoint in section 12 responds as expected	To be run on the final build

15. Future Enhancements

- Add login with JWT and separate roles for citizens and municipal officers.
- Add a `2dsphere` index and `$near` queries to find reports near a location.
- Send SMS or email messages when a status changes.
- Add a map and a dashboard front end.
- Allow a resolved report to be reopened, with a note in its history.
- Deploy the application to a hosting platform with the database address kept outside the code.

16. Conclusion

CivicFix shows the core MongoDB skills of this course on one working application: document modelling with embedded data, server-side validation, CRUD with an atomic status workflow, compound and text indexes checked with `explain()`, and aggregation pipelines including `$facet`.

A key design decision is the compound index of a field plus `created_at`. Every list query shows the newest reports first, so a single-field index was not enough. The compound index lets MongoDB find and order the reports in one step, and it reduced the documents examined for a city filter from 58 to 14.

The next stage should add user login, real data and deployment. With these, CivicFix could grow from an academic prototype into a useful tool for a municipal office.

References

1. MongoDB Documentation: Schema Validation, Indexes, Aggregation and explain(). https://www.mongodb.com/docs/manual/
2. PyMongo Documentation. https://pymongo.readthedocs.io/
3. FastAPI Documentation. https://fastapi.tiangolo.com/
4. CivicFix source code and sample data. https://github.com/Prashant-Baral/CivicFix

Appendix A. Project Structure

```
CivicFix/
  app/
    main.py          FastAPI entry point and startup
    config.py        Settings loaded from .env
    database.py      Connection, validation rule, 6 indexes
    models.py        Pydantic models and allowed values
    routes/
      issues.py      /issues endpoints
      analytics.py   /analytics endpoints
    services/
      crud.py        MongoDB queries and atomic status change
      analytics.py   Aggregation pipelines and explain()
  data.csv           58 sample reports
  requirements.txt   fastapi, uvicorn, pymongo, pydantic-settings
  .env.example       MONGODB_URI, MONGODB_DATABASE
  README.md          Setup and project notes
```

Appendix B. Academic Details

Field	Value
University	Lovely Professional University
School	School of Computer Science & Engineering
Student	Prashant Baral
Registration number	12327401
Program	B.Tech CSE
Roll number	65
GitHub URL	https://github.com/Prashant-Baral/CivicFix